

# AI Assisted Coding Assignment – 8.5

Name : G.Vamshi

Batch : 21

Ht.No : 2303A51448

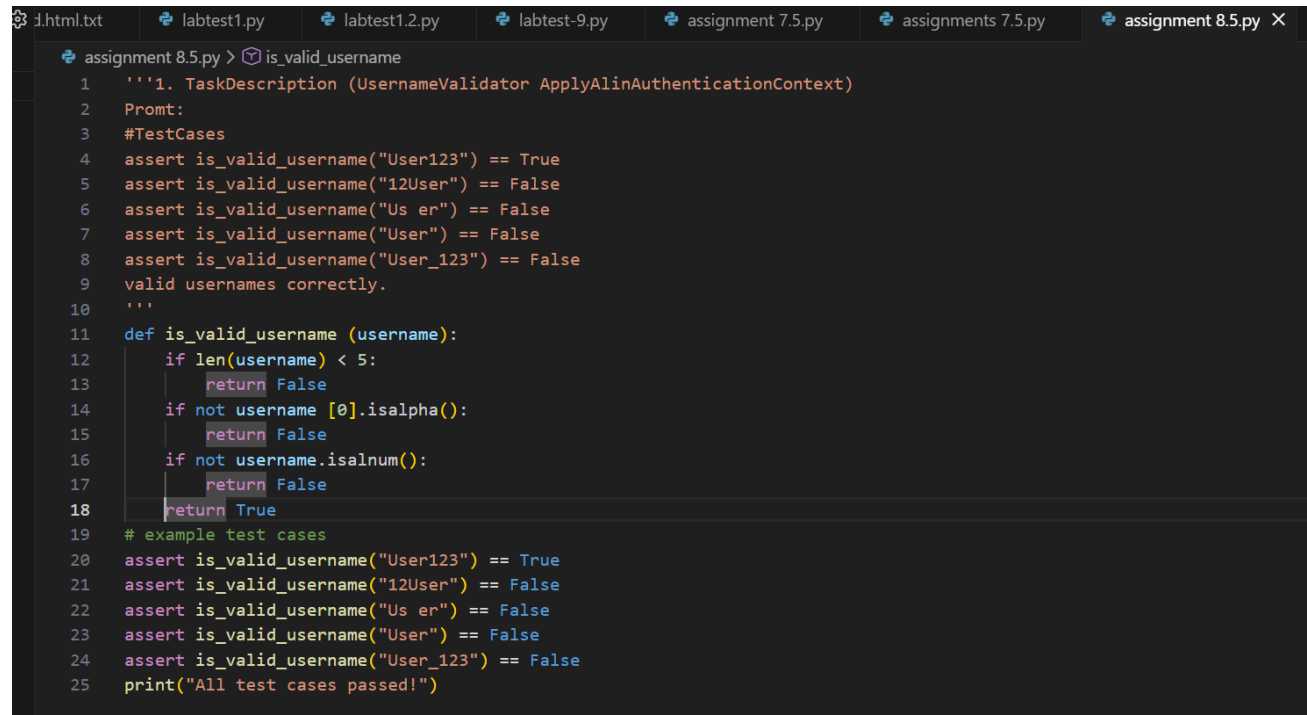
## Task 1 :

### TaskDescription(UsernameValidator–ApplyAlinAuthenticationContext)

**Prompt:** #TestCases

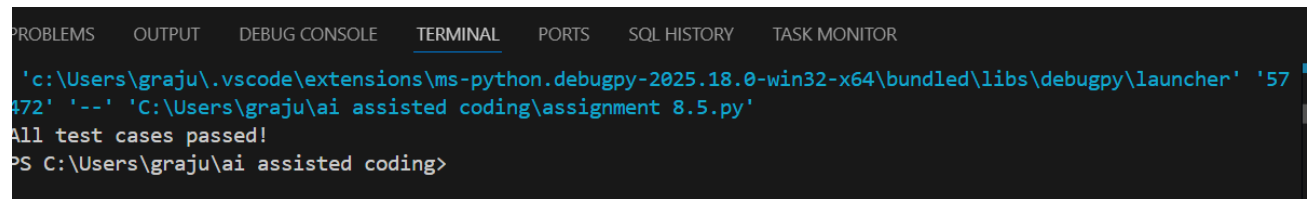
```
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
assert is_valid_username("User") == False
assert is_valid_username("User_123") == False
valid usernames correctly.
```

Code:

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'd.html.txt', 'labtest1.py', 'labtest1.2.py', 'labtest-9.py', 'assignment 7.5.py', 'assignments 7.5.py', and 'assignment 8.5.py'. The active tab is 'assignment 8.5.py'. The code in the editor is a Python script for a username validator. It starts with a docstring that includes the task description and test cases. Then, it defines a function 'is\_valid\_username' that checks if a username is valid based on length, first character, and character type. Finally, it includes example test cases and a print statement.

```
assignment 8.5.py > is_valid_username
1  '''1. TaskDescription (UsernameValidator ApplyAlinAuthenticationContext)
2  Prompt:
3  #TestCases
4  assert is_valid_username("User123") == True
5  assert is_valid_username("12User") == False
6  assert is_valid_username("Us er") == False
7  assert is_valid_username("User") == False
8  assert is_valid_username("User_123") == False
9  valid usernames correctly.
10 '''
11 def is_valid_username (username):
12     if len(username) < 5:
13         return False
14     if not username [0].isalpha():
15         return False
16     if not username.isalnum():
17         return False
18     return True
19 # example test cases
20 assert is_valid_username("User123") == True
21 assert is_valid_username("12User") == False
22 assert is_valid_username("Us er") == False
23 assert is_valid_username("User") == False
24 assert is_valid_username("User_123") == False
25 print("All test cases passed!")
```

## Output:



```
'c:\Users\graju\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57472' '--' 'C:\Users\graju\ai assisted coding\assignment 8.5.py'
All test cases passed!
PS C:\Users\graju\ai assisted coding>
```

## Observation:

AI-generated assert test cases helped define the user's name validation rules before coding. By writing tests first, the function was implemented to satisfy all constraints such as length limits, allowed characters, and starting character rules. This ensured the function was reliable and handled invalid usernames correctly.

## 2.Task: Even–Odd&TypeClassification

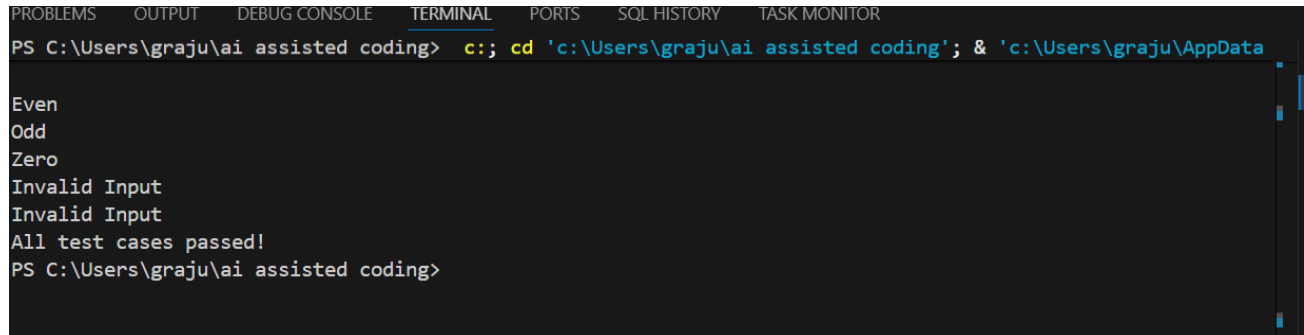
prompt:

```
assertclassify_value(8)=="Even"
assert classify_value(7) == "Odd"
assertclassify_value(0) == "Zero"
assertclassify_value("abc")=="InvalidInput"
assert classify_value(2.5) == "Invalid Input"
```

Code:

```
l.html.txt  labtest1.py  labtest1.2.py  labtest-9.py  assignment 7.5.py
assignment 8.5.py > classify_value
26  '''2. Task: Even Odd&TypeClassification
27  prompt:
28  assertclassify_value(8)=="Even"
29  assert classify_value(7) == "Odd"
30  assertclassify_value(0) == "Zero"
31  assertclassify_value("abc")=="InvalidInput"
32  assert classify_value(2.5) == "Invalid Input"
33  ...'''
34  def classify_value(value):
35      if isinstance(value, int):
36          if value == 0:
37              return "Zero"
38          elif value % 2 == 0:
39              return "Even"
40          else:
41              return "Odd"
42      else:
43          return "Invalid Input"
44  # example test cases
45  print(classify_value(8))
46  # "Even"
47  print(classify_value(7))
48  # "Odd"
49  print(classify_value(0))
50  # "Zero"
51  print(classify_value("abc"))
52  # "Invalid Input"
53  print(classify_value(2.5))
54  # "Invalid Input"
```

## Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR
PS C:\Users\graju\ai assisted coding> c++; cd 'c:\Users\graju\ai assisted coding'; & 'c:\Users\graju\AppData

Even
Odd
Zero
Invalid Input
Invalid Input
All test cases passed!
PS C:\Users\graju\ai assisted coding>
```

## Observation:

AI-generated assert test cases helped define the username validation rules before coding. By writing tests first, the function was implemented to satisfy all constraints such as length limits, allowed characters, and starting character rules. This ensured the function was reliable and handled invalid usernames correctly.

## Task3:PalindromeChecker

### Prompt:

```
assertis_palindrome("Madam")==True
```

```
assertis_palindrome("AmanaplanacanalPanama")==True assert
is_palindrome("Python") == False
assert is_palindrome("") == True
assertis_palindrome("a")==True
```

Code:

```
assignment 8.5.py > is_palindrome

56
57 '''Task3:PalindromeChecker
58 Prompt:
59 assertis_palindrome("Madam") == True
60 assertis_palindrome("AmanaplanacanalPanama") == True assert
61 is_palindrome("Python") == False
62 assert is_palindrome("") == True
63 assertis_palindrome("a") == True '''
64
65
66 def is_palindrome(s):
67     cleaned = ''.join(c.lower() for c in s if c.isalnum())
68     return cleaned == cleaned[::-1]
69 # example test cases
70 print(is_palindrome("Madam"))
71 print(is_palindrome("AmanaplanacanalPanama"))
72 print(is_palindrome("Python"))
73 print(is_palindrome(""))
74 print(is_palindrome("a"))
75 print("All test cases passed!")
76
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR
PS C:\Users\graju\ai assisted coding> & 'c:\Users\graju\AppData\Local\Programs\Python\Python314\python3.14t
.exe' 'c:\Users\graju\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher
True
True
False
True
True
All test cases passed!
```

Observation:

AI-generated tests helped identify edge cases like spaces, punctuation, and case differences. String normalization techniques were applied to ensure accurate palindrome detection. The function successfully handled empty strings and single-character inputs.

#### **Task4 : Observation:Bank Account Class**

**Prompt:**

```
acc=BankAccount(1000)
acc.deposit(500)
assert acc.get_balance() == 1500
```

```
acc.withdraw(300)
assert acc.get_balance() == 1200
```

```
acc.withdraw(2000)
assert acc.get_balance() == 1200
```

Code:

```
assignment 8.5.py > ...
26
27     '''Task4 : Observation: Bank Account Class
28     Prompt:
29     acc=BankAccount(1000)
30     acc.deposit(500)
31     assert acc.get_balance() == 1500
32     acc.withdraw(300)
33     assert acc.get_balance() == 1200
34     acc.withdraw(2000)
35     assert acc.get_balance() == 1200
36     '''
37     class BankAccount :
38         def __init__(self, initial_balance=0):
39             self.balance = initial_balance
40
41         def deposit(self, amount):
42             if amount > 0:
43                 self.balance += amount
44
45         def withdraw(self, amount):
46             if 0 < amount <= self.balance:
47                 self.balance -= amount
48
49         def get_balance(self):
50             return self.balance
51     # example test cases
52     acc = BankAccount(1000)
53     acc.deposit(500)
54     assert acc.get_balance()
55     acc.withdraw(300)
56     assert acc.get_balance()
57     acc.withdraw(2000)
58     assert acc.get_balance() == 1200
59     print("All test cases passed!")
```

## Output:

```
18.0-win32-x64\bundled\libs\debugpy\launcher' '49707' '--' 'C:\Users\graju\ai assisted coding\assignment 8.5'
.py'
.py'
All test cases passed!
PS C:\Users\graju\ai assisted coding> ...
```

## Observation:

AI-generated test cases helped design object-oriented methods before implementation. The class correctly handled deposits, withdrawals and balance retrieval. Test-driven development ensured correct behavior and reduced logical errors in financial operations.

## Task5:EmailIDValidation

### Prmot:

```
assertvalidate_email("user@example.com")==True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
assert validate_email("user@.com") == False
assertvalidate_email("user@gmail")==False
```



Code:

```
Intitled-1 | fronted.html.txt | labtest1.py | labt | Python
assignment 8.5.py > ...
1  '''Task5: EmailIDValidation
2  Prmot:
3  assert validate_email("user@example.com") == True
4  assert validate_email("userexample.com") == False
5  assert validate_email("@gmail.com") == False
6  assert validate_email("user@.com") == False
7  assert validate_email("user@gmail") == False'''
8
9  from ast import pattern
10 import re
11 def validate_email(email):
12     pattern = r'^[\w\.- ]+@[\w\.- ]+\.\w+$'
13     return re.match(pattern, email) is not None
14 # example test cases
15 print(validate_email("user@example.com"))
16 print(validate_email("userexample.com"))
17 print(validate_email("@gmail.com"))
18 print(validate_email("user@.com"))
19 print(validate_email("user@gmail"))
20 print("All test cases passed!")
21
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
True
False
False
False
False
All test cases passed!
```

**Observation:**

AI test cases guided the validation rules for email format. The function

correctly checked for required symbols and invalid formats. Edge cases such as missing symbols and improper formats were handled effectively, improving data validation reliability